DESENVOLVIMENTO DE APLICATIVOS MOBILE · MÓDULO 14

Tópicos Avançados e Projeto Final

IA no dispositivo e na nuvem, segurança, monetização, outras telas, carreira e o projeto integrador

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Integrar modelos de aprendizado de máquina no app, no aparelho e na nuvem.
- Aplicar as principais práticas de segurança mobile (OWASP MASVS).
- Conhecer os modelos de monetização e as APIs da Google Play.
- Levar o app a widgets, relógios e carros.
- Planejar, executar e apresentar o projeto final do curso.

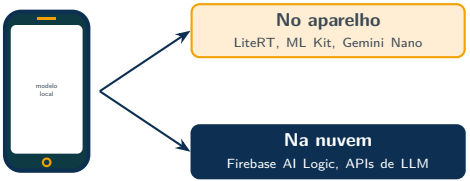
Roteiro

Neste módulo

- Inteligência artificial no app
- Segurança mobile
- Monetização e outras telas
- Carreira e portfólio
- Projeto final
- Fechamento do curso

Inteligência artificial no app

Onde roda o modelo?



	Aparelho	Nuvem
Latência	muito baixa	depende da rede
Offline	sim	não
Privacidade	dados ficam no aparelho	dados saem
Custo	nenhum por uso	por requisição
Tamanho do modelo	pequeno (KB–MB)	grande

ML Kit

Modelos prontos do Google: texto em imagem (OCR), códigos de barras, rostos, rótulos, tradução.

LiteRT

Novo nome do **TensorFlow Lite**: roda o **seu** modelo (`.tflite`) com aceleração por GPU/NPU.

Gemini Nano

LLM do sistema em aparelhos compatíveis, acessível pelas APIs de IA generativa do ML Kit.

Detectando anomalias nas leituras

1) Linha de base estatística (z-score)

```
fun Flow<Double>.anomalias(
  janela: Int = 60, limiar: Double = 3.0
): Flow<Double> = flow {
  val buf = ArrayDeque<Double>()
  collect { x ->
    if (buf.size == janela) {
      val m = buf.average()
      val dp = sqrt(buf.sumOf { (it - m).pow(2) } /
      janela)
      if (dp > 0 && abs(x - m) / dp > limiar) emit(x)
      buf.removeFirst()
    }
    buf.addLast(x)
  }
}
```

2) Modelo próprio com LiteRT

```
class DetectorAnomalia(ctx: Context) {
  private val modelo = Interpreter(
    FileUtil.loadMappedFile(ctx, "anomalia.tflite"))

  // janela de 60 leituras normalizadas -> prob.
  fun probabilidade(janela: FloatArray): Float {
    val entrada = arrayOf(janela)          // [1, 60]
    val saida = Array(1) { FloatArray(1) } // [1, 1]
    modelo.run(entrada, saida)
    return saida[0][0]
  }
}
```

Treino em Python (Keras) e conversão

```
conv = tf.lite.TFLiteConverter.from_keras_model(m)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
open("anomalia.tflite", "wb").write(conv.convert())
```

Dica

Sempre compare o modelo de IA com uma linha de base simples como esta. Às vezes ela basta.

IA na nuvem e boas práticas

Firebase AI Logic

SDK para chamar modelos Gemini a partir do app **sem expor chaves**: o Firebase intermedeia a chamada e o **App Check** garante que só o seu app usa a cota.

Ideias para o LabMonitor

- "Explique este alerta" a partir das últimas leituras.
- Fotografar a placa do equipamento e preencher o cadastro (OCR).
- Resumo semanal de uso das bancadas em linguagem natural.

Cuidados

- LLMs erram: mostre como sugestão e peça confirmação.
- Dados pessoais só com necessidade e aviso (LGPD).
- Limite o uso e trate falhas da rede.
- Quantize modelos locais (*int8*).

Dica

Teste o modelo no aparelho mais fraco do seu público.

Segurança mobile

OWASP MASVS: os pilares

Área	Práticas no Android
Armazenamento	dados sensíveis só no armazenamento interno; chaves no **Android Keystore**; nada sensível em logs ou backups
Criptografia	algoritmos padrão (AES-GCM, RSA/EC) via Keystore; nunca criptografia "caseira"
Autenticação	tokens de curta duração; biometria com `BiometricPrompt`; *passkeys* com Credential Manager
Rede	HTTPS sempre; *network security config*; *pinning* só com plano de rotação
Plataforma	expor o mínimo de componentes; validar *intents* e *deep links*; WebView sem JavaScript desnecessário
Código	dependências atualizadas; R8; nenhum segredo no APK
Resiliência	**Play Integrity API** para verificar app e aparelho no servidor

O cliente nunca é confiável: toda regra importante precisa ser verificada também no servidor.

Monetização e outras telas

Modelos de monetização

Modelo	Como funciona
Pago	usuário paga para baixar
Freemium	grátis com recursos pagos (compras no app)
Assinatura	cobrança recorrente por acesso
Anúncios	receita por exibição (AdMob)
B2B / institucional	a organização paga pelo app e pelo serviço
Gratuito	projetos acadêmicos, públicos e de extensão

Google Play Billing

Bens digitais vendidos dentro de apps distribuídos pela Play usam o sistema de faturamento da Play, que retém uma taxa de serviço. Bens físicos e serviços prestados fora do app seguem outras regras.

Atenção

Valide as compras **no servidor** com a API da Play antes de liberar o conteúdo.

Dica

Para apps institucionais, o modelo comum é gratuito, sem anúncios, com o custo coberto pelo projeto.

Além do celular



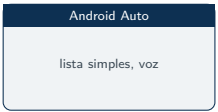
Widgets (Glance)

Informação na tela inicial, escrita com uma API no estilo Compose.



Wear OS

Apps e *tiles* no relógio com Compose for Wear OS; ideal para alertas rápidos.



Carro e TV

Android Auto, Automotive e TV usam *templates* e regras de segurança próprias.

Dica

A arquitetura em camadas permite reaproveitar repositórios e casos de uso em todas essas superfícies; só a camada de UI muda.

Carreira e portfólio

Do curso ao mercado

Portfólio que convence

- 2–3 apps completos no GitHub, com README, capturas e arquitetura explicada.
- Pelo menos um app publicado na Play (mesmo em teste aberto).
- Testes e CI visíveis no repositório.
- Um projeto com hardware (BLE/IoT) é um diferencial raro.

Continue aprendendo

- Código-fonte do *Now in Android* e dos *samples* oficiais.
- Blog *Android Developers* e as notas de versão do Jetpack.
- Comunidades: GDG, Kotlin User Groups, eventos locais.
- Contribua com bibliotecas de código aberto.

Assistentes de IA

Use-os para acelerar, mas entenda e revise cada linha: você responde pelo código que publica.

Projeto final

Projeto final: requisitos

Requisitos mínimos

1. 4+ telas com Navigation 3 e Material 3.
2. MVVM com repositórios e Hilt.
3. Room e DataStore.
4. API REST ou Firebase.
5. Offline nas telas principais.
6. Um recurso do dispositivo (BLE, câmera...).
7. Testes de ViewModels e casos de uso.
8. Acessibilidade e tema escuro.

Sugestões de tema

- Concluir e publicar o **LabMonitor**.
- Controle de empréstimo de equipamentos do laboratório via QR code.
- Medidor de consumo de energia com ESP32 e sensor de corrente.
- App de acompanhamento de estágio ou TCC.

Dica

Resolva um problema real de alguém que você conhece.

Entregáveis, cronograma e avaliação

Semana	Marco
1	proposta: problema, público, telas (protótipo) e arquitetura
2	navegação, UI e dados locais
3	rede/Firebase, recurso do dispositivo e offline
4	testes, acessibilidade, release e apresentação

Entregáveis

Repositório GitHub com README, APK/AAB de release, vídeo de 3 minutos e apresentação de 10 minutos.

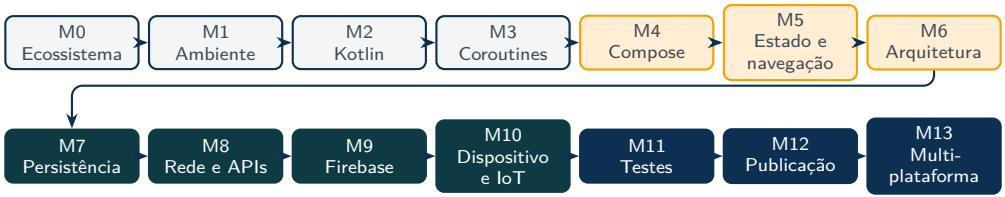
Critério de avaliação	Peso
Funcionalidades e requisitos atendidos	25 %
Arquitetura e qualidade do código	20 %
Experiência do usuário, UI e acessibilidade	15 %
Testes automatizados e CI	15 %
Uso de recursos do dispositivo / integração	10 %
Documentação (README, diagramas)	5 %
Apresentação e demonstração	10 %
Total	**100 %**

Dica

Pesos sugeridos: ajuste conforme o plano de ensino da disciplina.

Fechamento do curso

A jornada completa



O que você construiu

Um app Android completo e publicado, com arquitetura profissional, dados locais e remotos, integração com hardware, testes e entrega contínua, além da base para levá-lo a outras plataformas.

Exercícios finais

Exercício 1 — IA

Gere 24 h de leituras simuladas com picos artificiais e compare o detector por z-score com um modelo treinado em Python e convertido para LiteRT: quantos picos cada um encontra?

Exercício 2 — Segurança

Faça uma revisão do LabMonitor com o checklist MASVS do slide e registre ao menos três correções em *issues* no GitHub.

Exercício 3 — Widget

Crie um widget Glance que mostre a última temperatura da bancada favorita e abra o app ao ser tocado.

Projeto final

Escolha o tema, entregue a proposta na semana 1 e siga o cronograma de 4 semanas.

Para aprofundar

- IA no Android: `https://developer.android.com/ai`
- LiteRT: `https://ai.google.dev/edge/litert` ML Kit: `https://developers.google.com/ml-kit`
- Firebase AI Logic: `https://firebase.google.com/docs/ai-logic`
- OWASP MASVS: `https://mas.owasp.org/MASVS/`
- Segurança no Android: `https://developer.android.com/privacy-and-security/security-tips`
- Google Play Billing: `https://developer.android.com/google/play/billing`
- Glance (widgets): `https://developer.android.com/develop/ui/compose/glance`

Obrigado!

Prof. Dr. Francisco dos Santos Viana

Fim do curso Desenvolvimento de Aplicativos Mobile